01 Introduction to Robotics

WHAT IS ROBOTICS?

Why it matters: Before you build a robot, you need to know what makes something a robot and not just a machine.

Robotics deals with the design, construction, operation, and use of robots. It also covers the computer systems that control them, give them sensory feedback, and process their information.

A robot is an automatically operated machine that replaces human effort. Two things set a robot apart:

- It can perform autonomously, without a person controlling every move.
- It can carry out repetitive instructions with precision and accuracy, doing the same task many times and doing it well each time.

Notice that precision and accuracy show up in the definition of a robot itself. That's why they're one of Acamad's Technical Standards (T04).

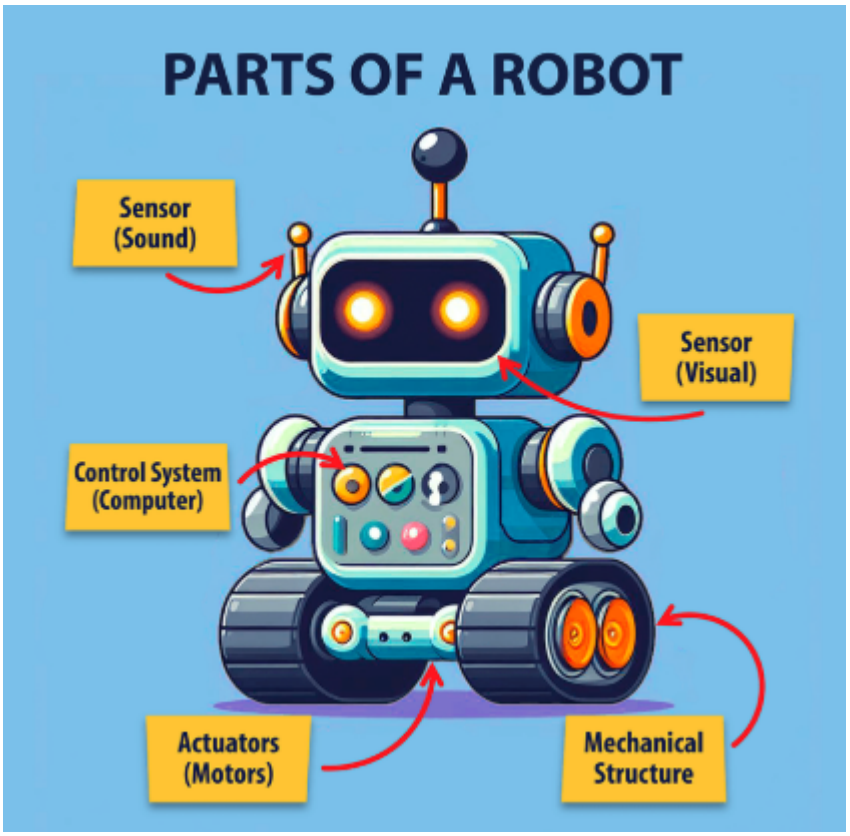
BASIC PARTS OF A ROBOT

Every robot, from a classroom line tracer to a factory arm, is built from the same four groups of parts. If you know the groups, you know where to look when something goes wrong.

Part	What it does	Think of it as
Mechanical	The parts that move	The body
Actuation	Moves the mechanical parts (e.g., motors)	The muscles
Sensory	Connects the robot to its environment	The senses
Control	Processes information and makes decisions	The brain

These four groups depend on each other. The sensors gather information, the controller decides what to do with it, and the actuators move the mechanical parts. If one group fails, the whole robot fails. That's the idea of integration you'll see again in T03 (Functionality).

Example: A line-tracing robot uses sensors to detect the line, a controller to decide which way to steer, motors (actuation) to turn the wheels, and a chassis and wheels (mechanical) to move across the field.



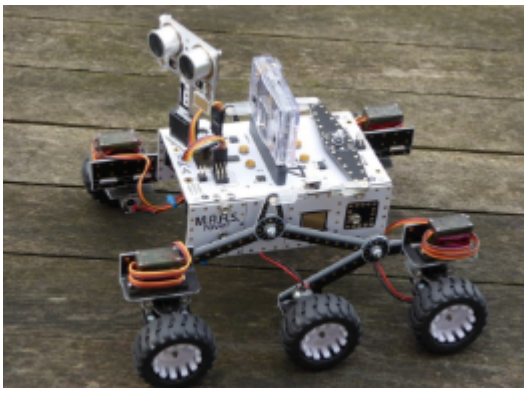
TYPES OF ROBOTS

Where a robot works decides how it should be built.

- Industrial robots are used in factories to automate production lines. They operate at high precision and speed.



- Field robots are non-factory robots, usually mobile, that operate in complex and changing environments.



- Service robots work closely with humans to help them in their daily lives.



ROBOTICS PLATFORMS AND COMPETITIONS

Knowing the platforms and competitions helps you see what kind of robot you'll be building, and with what tools.

Common robotics platforms:

- Arduino
- INEX ATX
- VEX
- LEGO
- Roborobo



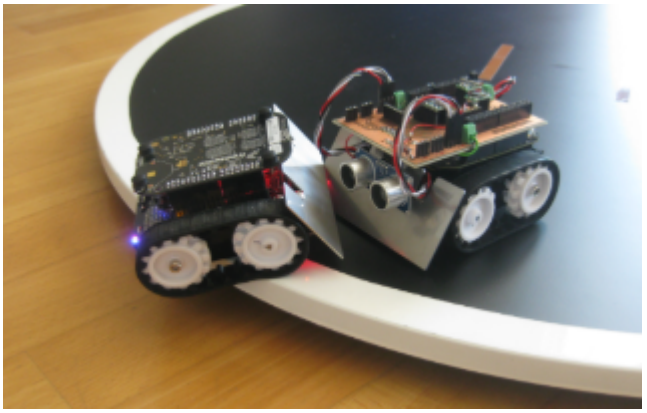


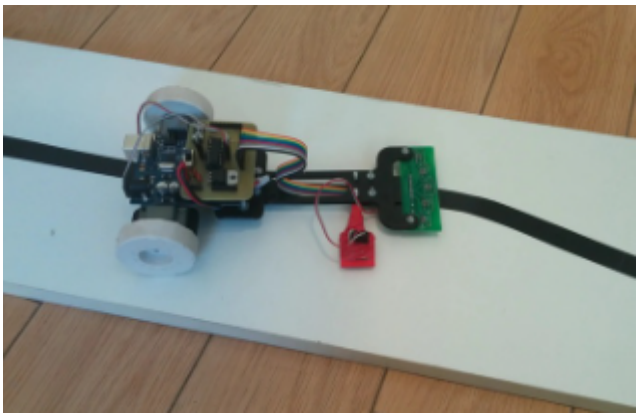


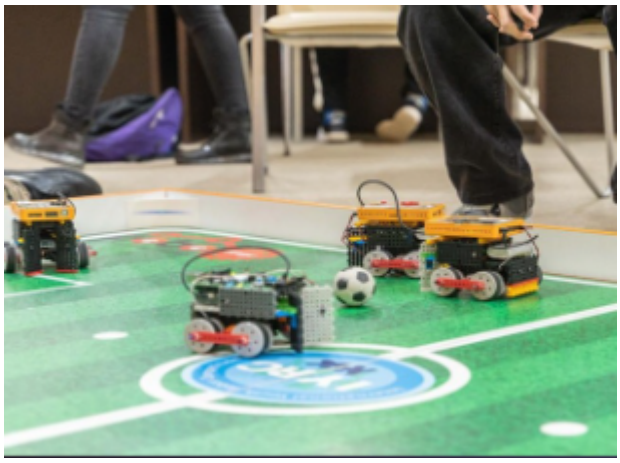
Common robotics competitions:

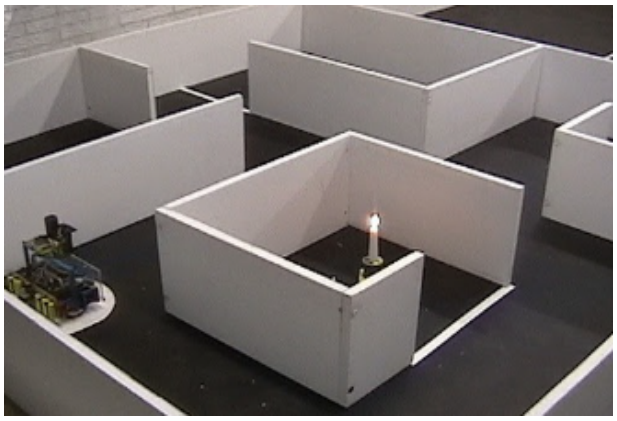
- Sumobot: robots try to push each other out of a ring
- Line Tracing: robots follow a line on the field
- Mission Challenge: robots complete a set of tasks on a field
- Innovative Challenge: teams build a project that solves a problem (the Creative side)
- Firefighting: robots find and put out a flame
- Robot Soccer: robots play a soccer match

Most of these are Technical competitions, judged by how well the robot performs. The Innovative Challenge belongs to the Creative side. You'll learn the standards for both in Section 02.









02 Acamad's Robotics Standards

UNDERSTANDING COMPETITIVE ROBOTICS

The Two Sides of Competitive Robotics

When people think of robotics competitions, they usually picture robots racing, pushing, or picking up objects on a field. That's one side. But there's another side just as important.

The Technical side is about performance and engineering. These are the traditional robot games: a field, a set of tasks, a scoring system, and robots built to complete those tasks as well as possible. The question here is: How well does this robot perform?

The Creative side is about innovation and research. Here, you design a robotics project that solves a real problem or explores a new possibility. The question here is: Is this idea worth building, and is it built on solid thinking?

These two sides are not separate worlds. A strong creative project still needs good engineering to work, and technical teams still need creative thinking to design clever solutions. They overlap and support each other.

What is a Standard?

A standard is a defined criterion used to judge the quality or performance of robotics work. Instead of saying "this robot looks good" or "this project seems cool," standards give you specific things to check.

Each side has five standards:

Technical Standard:

1. Proportionality
2. Stability & Balance
3. Functionality
4. Precision & Accuracy
5. Consistency

Creative Standard:

1. Value
2. Feasibility
3. Credibility
4. Logic
5. Consistency

Why the Order Matters?

The standards are sequential. Each one builds on the ones before it. Think of it like building a house: you don't paint the walls before the foundation is poured.

On the Technical side:

- If materials are badly distributed (Proportionality), weight ends up uneven.
- If weight is uneven (Stability), the robot tips or drifts, and it can't do its job.
- If it can't do its job (Functionality), there's no point measuring how exactly it does it.
- If it isn't exact (Precision & Accuracy), it can't be reliable.
- Only after all that can you ask: does it do this every single time? (Consistency)

Notice that Consistency is last on both sides. It's the final proof that the work is reliable after everything before it is in place.

One more key idea: evaluate work using the right standards. Don't judge a creative research project only by how fast its robot drives, and don't judge a competition robot by how original the idea is.

Common Mistakes

- "Technical and Creative robotics are completely separate." → They overlap and support each other.
- "A robot only needs to work to be successful." → Working is only one standard out of five.
- "A project is innovative just because the idea is new." → A new idea still has to prove its value and pass the Creative standards.
- "I can check the standards in any order." → Later standards depend on earlier ones.

T01 - PROPORTIONALITY

Why it matters: How you place parts on your robot decides almost everything that comes after: its weight, its balance, and how well it performs.

Proportionality is the equal or appropriate distribution of materials throughout a robot, according to its design requirements.

Two words in that definition are worth slowing down for:

- Materials means the physical parts and substances used to build the robot: the battery, motors, controller, frame, and so on.
- Appropriate means "right for this robot's purpose." It does not mean "identical."

A proportional robot doesn't need every part to be the same size, amount, or shape. A robot built to lift objects will place its parts differently from a robot built for speed, and both can be proportional. What matters is that the distribution makes sense for the design.

Proportionality also means no unnecessary material. If a part doesn't help the robot do its job, it's adding weight and taking up space for nothing.

Tool: The Quadrant Check

1. Look at your robot from the top view.
2. Imagine two perpendicular lines crossing at the center of the chassis. This splits it into four quadrants: front-left, front-right, back-left, back-right.
3. List what components sit in each quadrant.
4. For each component on one side of a line, look at the opposite side. Is there something that keeps the distribution appropriate?
5. If one quadrant holds a big component and the opposite quadrant is empty, stop and ask: Does the design actually need it this way?

Example: Your battery sits in the front-left quadrant. Now look at the front-right. Is it empty? Does that make sense for your robot, or should something be rearranged?

Important: The Quadrant Check is a guideline, not a rule. Components don't have to be identical or perfectly mirrored. Sometimes an uneven layout is intentional. The point is that you chose it for a reason.

Key terms

- Design requirement: a condition the robot's design must satisfy.
- Distribution: how materials or components are arranged throughout the robot.
- Material: a physical substance or component used to build the robot.

Common mistakes

- "Proportional means every part is the same size."
- "Parts must be identical or symmetrical on opposite sides."
- "Proportionality and Stability are the same thing." → Proportionality is about materials. Stability is about weight. You'll see the difference next lesson.

T02 - STABILITY & BALANCE

Why it matters: A robot that tilts, wobbles, or tips over can't complete tasks, no matter how good its code is.

Stability & Balance is about the appropriate distribution of weight throughout the robot.

Here's how this connects to Proportionality: Proportionality asks where are the materials? Stability asks where is the weight? Two parts of the same size can weigh very different amounts. A battery and a sensor might take up similar space, but the battery is much heavier. So a robot can look evenly laid out and still be badly balanced.

The Center of Gravity (CoG)

The Center of Gravity is the point where the robot's total weight can be considered to act. It's like the robot's "balance point."

Key facts about the CoG:

- It sits closer to where most of the mass is. Heavy parts pull it toward them.
- It's not automatically the center of the chassis. It depends on the mass and placement of every component.
- A higher CoG makes a robot more likely to tip or move in unwanted ways. Taller robots often have higher CoGs.

The Base of Support

The base of support is the area enclosed by the robot's points of contact with the ground (for example, its wheels). A wider base generally means more stability.

Think of how you stand on a moving bus. You naturally spread your feet apart and bend your knees. Spreading your feet widens your base of support. Bending your knees lowers your center of gravity. Robots follow the same idea.

Stability isn't only about standing still. Weight distribution matters most when the robot is moving: turning, accelerating, and stopping suddenly. A robot that's fine while parked can still tip during a sharp stop.

Counterweights are weights added on purpose to adjust weight distribution and improve balance.

Tool: Estimating the CoG

1. Identify heavy components: battery, motors, and other heavy parts.
2. Observe their placement: Is most of the weight on one side? Near the top?
3. Estimate the CoG: It will be closer to where the mass is concentrated.
4. Adjust: Move heavy parts lower and distribute weight better.

Example: If the battery and motors are both on the left side, the CoG is probably closer to the left. Moving some of that weight can improve balance.

Key terms

- Balance: weight distributed well enough to prevent unwanted tilting or tipping.
- Base of support: the area enclosed by the robot's contact points with the ground.
- Center of Gravity (CoG): where the robot's total weight can be considered to act.
- Stability: the robot's ability to stay controlled and steady during operation.
- Weight distribution: how the total weight is spread across the robot.

Common mistakes

- "Stability only matters when the robot isn't moving."
- "Putting everything in the middle automatically balances the robot." → Balance depends on overall weight, placement, and the CoG.
- "A robot with a heavy center is always stable." → It also depends on how the CoG relates to the base of support.
- "Height doesn't affect movement." → Taller robots can have higher CoGs.
- "The center of the chassis is always the CoG."

T03 - FUNCTIONALITY

Why it matters: This is where the robot proves it can actually do what it was built for.

Functionality is the robot's ability to perform its intended purpose and required tasks.

Three things have to be true for a robot to be functional:

1. Every component has a purpose. Each part contributes to the robot's functions.
2. Components are compatible. They're able to work together correctly.
3. Components are integrated. The mechanical, electrical, and software parts are connected so they work as one system.

A robot can have great parts and still fail if those parts don't work together. A strong motor is useless if the code never tells it what to do, or if it doesn't fit the mechanism it's supposed to drive.

"It moves" isn't the same as "it works." A robot rolling around the field proves the motors run. It doesn't prove the robot can push the block, grab the ring, or follow the line it was designed for.

"It worked once" isn't enough either. A robot is only functional when its functions hold up under the conditions it will actually operate in: the real field, the real surface, the real competition situation.

Key terms

- Compatibility: the ability of components or systems to work together correctly.

- Functionality: the ability to perform an intended purpose.
- Integration: connecting mechanical, electrical, and software components so they work together.

Common mistakes

- “If the robot moves, it’s functional.”
- “If it worked once, it’s fully functional.”

T04 - PRECISION & ACCURACY

Why it matters: In competitions, “close enough” often isn’t. Points depend on hitting exact positions and targets.

These two words are often used as if they mean the same thing. In robotics, they don’t.

- Accuracy is how close a result is to the intended target.
- Precision is how close repeated results are to each other.

The dartboard picture. Imagine throwing four darts:

What happens	Precise?	Accurate?
All four land in a tight group, right on the bullseye	✅	✅
All four land in a tight group, but in the corner of the board	✅	❌
One lands on the bullseye, the others scatter everywhere	❌	Only that once
All four scatter randomly	❌	❌

So:

- A robot can be precise without being accurate: it gets the same result every time, but consistently misses the target.
- A robot can be accurate once without being precise: it hits the target on one try, but its results vary across tries.

This is why one trial tells you almost nothing. You need repeated trials to know whether a robot is precise.

What affects precision and accuracy?

It isn’t just the code. Precision and accuracy can be affected by:

- Mechanical alignment
- Sensor readings

- Motor control
- Programming
- Environmental conditions (lighting, temperature, the surface the robot drives on)

Because the environment matters, test in conditions similar to where the robot will compete. Testing and calibration help you find and reduce these errors. (Calibration gets its own lesson next.)

Tool: The Precision & Accuracy Check

1. Choose a target, such as stopping at a marked line.
2. Run repeated trials.
3. Record the results. Measure in centimeters or millimeters.
4. Identify the issue:
    - Results close together but all missing the target → precise, not accurate
    - Results spread far apart → lacks precision
5. Adjust and retest. Look into programming, sensors, mechanical alignment, and the environment.

Example: A robot is programmed to stop at a line. It stops 2 cm before the line in every trial. That's precise, but not accurate. If it stopped at a different distance each time, its precision would be poor.

Key terms

- Accuracy: how close a result is to the intended or correct value.
- Calibration: adjusting or verifying a device's measurements or output against a known reference.
- Deviation: the difference between an observed result and the intended value.
- Error: the difference between the actual result and the expected result.
- Precision: how close repeated results are to one another.

Common mistakes

- "Precision and accuracy mean the same thing."
- "Hitting the target once means the robot is precise and accurate."
- "Programming alone controls precision and accuracy." → Mechanics, sensors, motors, calibration, and the environment matter too.
- "I only need to calibrate once."

T05 - CONSISTENCY

Why it matters: In a competition, you don't get to choose which run counts. Your robot has to perform when it matters, every time.

Consistency is the robot's ability to produce reliable and repeatable results across multiple attempts under similar conditions.

It comes last for a reason. Consistency is the proof that everything before it holds up: the layout, the balance, the function, and the precision.

Consistency doesn't mean perfection. Small variations are normal. The goal is performance that's repeatable enough for the task. If your robot needs to stop within a scoring zone, a few millimeters of difference between runs may not matter.

Where inconsistency comes from

It's tempting to blame the code every time. But inconsistent performance can come from four areas:

- Mechanical: loose screws, wobbly parts, misaligned wheels
- Electrical: loose or inconsistent connections
- Sensor-related: readings that change between runs
- Software: programming issues

Fix the cause, not the symptom. Recalibrating before every run might help for a while, but it hides the real problem. If something keeps drifting, find out why.

Tool: The Consistency Check

1. Choose a task you can repeat: drive a set distance, turn, pick up a game piece.
2. Repeat it several times with the same program and setup.
3. Observe. Does the robot do the same sequence of actions with similar results every time?
4. Identify variations in movement, timing, sensor responses, or mechanisms.
5. Investigate. Check for loose components, inconsistent connections, changing sensor readings, or code issues.
6. Retest after each adjustment.

Example: Your robot is programmed to drive forward and turn at a specific point. If it completes the sequence every time, it's consistent. If it turns correctly in some runs but misses the turn in others, investigate why.

Key terms

- Consistency: producing similar results repeatedly under similar conditions.
- Reliability: performing intended functions dependably over time.
- Repeatability: achieving similar results when the same procedure is repeated under the same conditions.
- Variation: a difference in results across attempts.
- Troubleshooting: identifying and resolving the causes of problems in a system.

Common mistakes

- “Inconsistency is always a programming problem.”
- “Recalibrating before every run is the only way to improve consistency.”
- “Consistency means identical results every time.”

03 Basic Electronics

Every robot runs on electricity. Before a robot can move, sense, or think, its circuits have to be built correctly. A single wrong connection can stop a robot or destroy a component.

VOLTAGE, CURRENT, AND RESISTANCE

Three quantities describe every circuit:

- Voltage (V): the electrical “push” that moves current through a circuit. Measured in volts (V).
- Current (I): the flow of electricity through a circuit. Measured in amperes (A).
- Resistance (R): how much a component limits the flow of current. Measured in ohms (Ω).

Think of water in a pipe. Voltage is the water pressure, current is how much water flows, and resistance is how narrow the pipe is. More pressure pushes more water. A narrower pipe lets less water through.

THE BREADBOARD (PROTOBOARD)

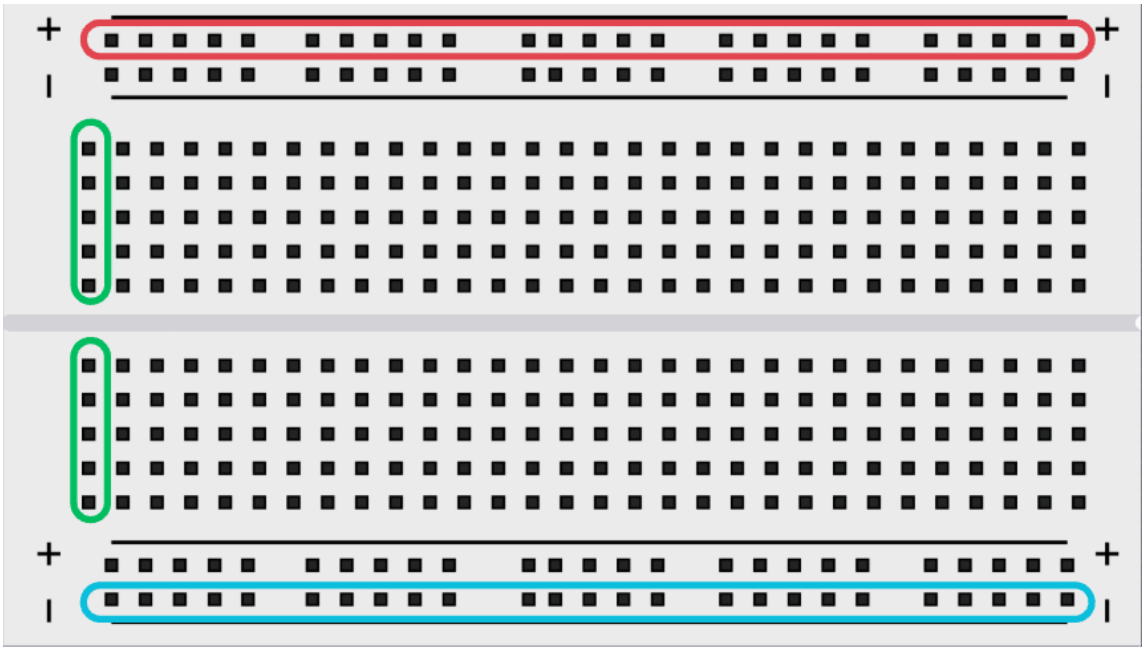
A breadboard is used for prototyping and testing electronic circuits. It lets you build and change circuits quickly, without soldering components together. You make connections using jumper wires.

How the holes are connected

The holes on a breadboard aren’t all separate. Inside, metal strips connect certain holes:

- Power rails: the long rows along the top and bottom edges, usually marked red (+) and blue (–). All the holes in one rail are connected horizontally, along the length of the board. Use them to share power and ground across your circuit.
- Body (Terminal strips): the main area in the middle. Holes are connected vertically in short groups of five. Each group is a separate connection.
- Center gap: the channel down the middle. It separates the top and bottom halves, so the groups on either side are not connected.

Example: If you put a resistor leg and an LED leg in the same vertical group of five, they’re connected. Move the LED one column over, and they’re not.



TYPES OF CIRCUITS

There are two basic ways to connect components:

Series circuit: components are connected one after another, in a single path. The same current flows through every component.

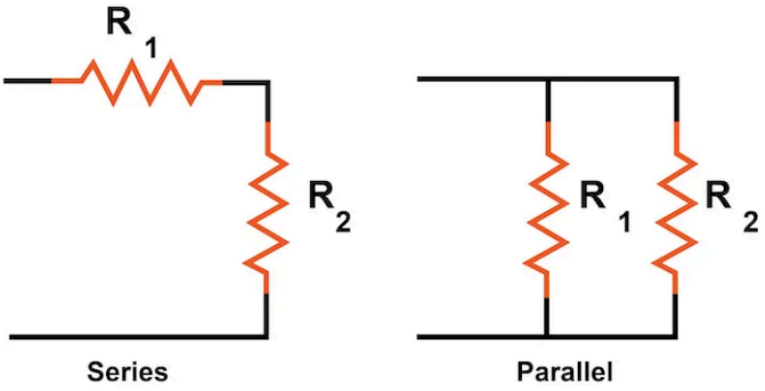
- If one component breaks or is removed, the path is broken and everything stops.
- Example: a battery, one resistor, and one LED connected in a single loop.

Parallel circuit: components are connected on separate branches, each with its own path back to the power source.

- If one branch breaks, the other branches keep working.
- Example: a 9 V battery powering three branches, each with its own resistor and LED.

Tool: Trace the Path

Put your finger on the positive terminal of the power source and follow the wire. If there's only one route back to negative, it's series. If the route splits into branches, it's parallel.



RESISTORS

Without a resistor, too much current can rush through a component and destroy it.

A resistor is a two-terminal electrical component that limits the flow of current. It controls the voltage and current in a circuit, which keeps components like LEDs from being damaged by too much current.

Resistance is measured in ohms (Ω). A resistor's value is shown using colored bands. Each color represents a number.

Tool: Reading a 4-Band Resistor

Hold the resistor so the gold or silver band is on the right. Read from left to right:

1. Band 1: first digit
2. Band 2: second digit
3. Band 3: multiplier (how many zeros to add)
4. Band 4: tolerance (how far the actual value may vary)

Example: Brown, Red, Red, Gold

- Brown = 1, Red = 2 → 12
- Red multiplier = × 100 → 12 × 100 = 1,200 Ω
- Gold = ±5%
- Value: 1,200 Ω ±5% (also written 1.2 kΩ)

Note: Most resistors have 4 bands. For almost all of your circuits, you can ignore the 4th (tolerance) band. Some resistors have 5 bands: three digit bands, then the multiplier, then the tolerance.

Color	1st Digit	2nd Digit	3rd Digit		Multiplier	Tolerance
Black	0	0	0		1 Ω	
Brown	1	1	1		10 Ω	± 1%
Red	2	2	2		100 Ω	± 2%
Orange	3	3	3	×	1k Ω	
Yellow	4	4	4		10k Ω	
Green	5	5	5		100k Ω	± 0.5%
Blue	6	6	6		1M Ω	± 0.25%
Violet	7	7	7			± 0.1%
Grey	8	8	8			
White	9	9	9			
Gold					0.1 Ω	± 5%
Silver					0.01 Ω	± 10%

LIGHT-EMITTING DIODE (LED)

An LED is a type of diode that emits light when current flows through it.

Polarity: LEDs only work one way

An LED has two legs, and they're not interchangeable:

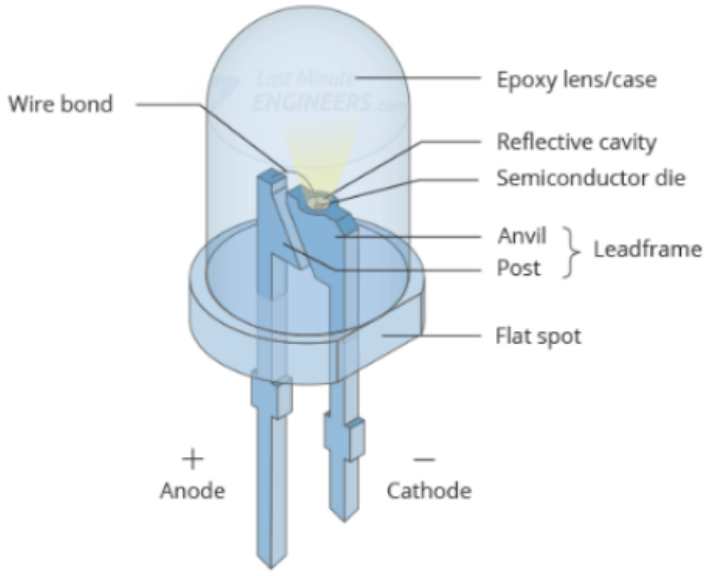
- Anode (+): the longer leg. It connects toward the positive side.
- Cathode (–): the shorter leg. It connects toward the negative side. The cathode side of the LED's rim also has a flat spot.

If you connect an LED backward, it won't light up.

Safety rules

- Always connect an LED with a resistor in series.
- NEVER connect an LED directly to a battery or power supply. Without a resistor, too much current flows and the LED can burn out.

Uses: status indicators, digital displays, lighting systems



POTENTIOMETER (VARIABLE RESISTOR)

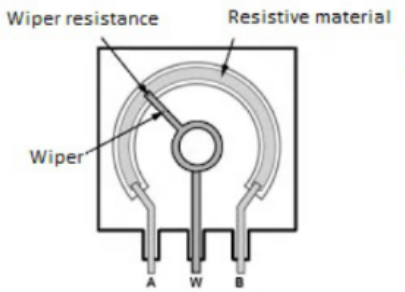
A potentiometer is a three-terminal resistor that lets you adjust its resistance by hand. Turning the knob moves a wiper along a strip of resistive material. That changes the resistance, which changes the current flow.

Uses: controlling the volume of a speaker, the brightness of an LED

Example: Connect a potentiometer in series with an LED and resistor. Turn the knob one way, and resistance goes up, so current goes down and the LED dims (Ohm's Law: I = V ÷ R). Turn it the other way, and the LED brightens.



Image of a Potentiometer



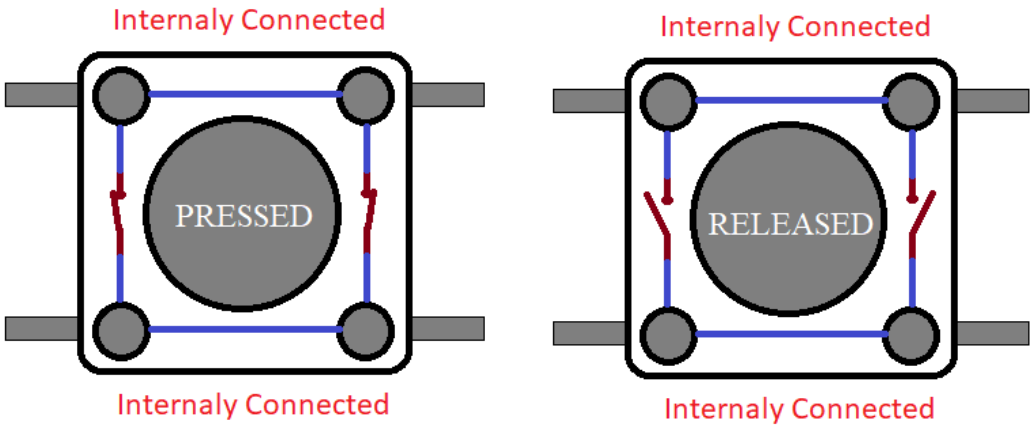
Internal structure of a Pot

PUSH BUTTONS

A push button is a switch that opens or closes a circuit. There are two types.

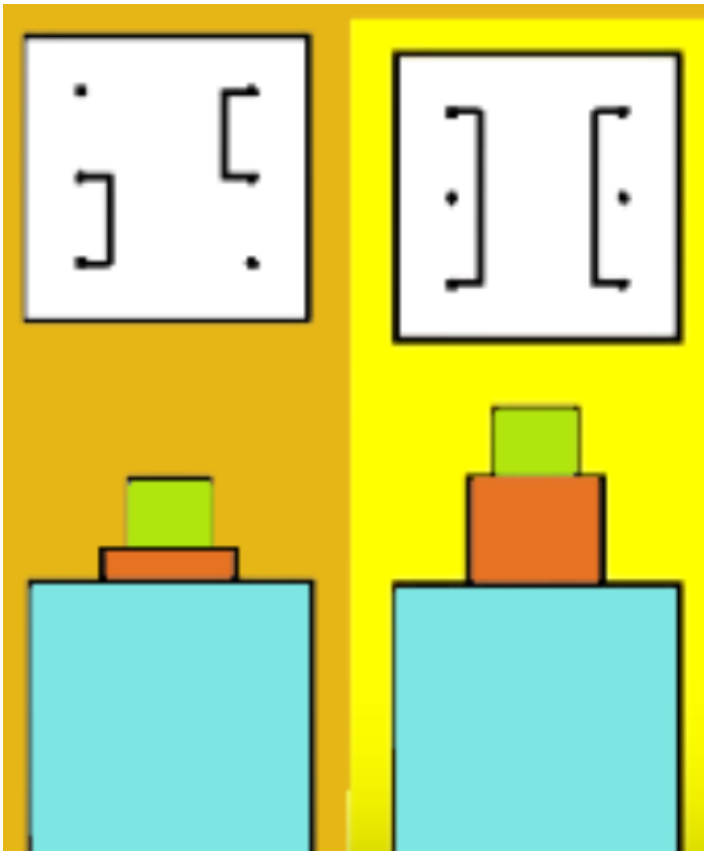
Momentary push button: only stays on while being pressed. When you release it, it goes back to its default off position.

- Pressed: completes the circuit (closes the switch), so current flows.
- Unpressed: opens the circuit, so current stops.
- Uses: control systems (keyboards, doorbells), start/stop functions, user interactions



Self-locking push button: each press toggles between two states, and it stays in the new state until you press it again.

- On: locked
- Off: unlocked
- Uses: toggle switches (light switches), safety systems (fire alarms, elevator buttons)



Tool: Which Button Do I Need?

Ask: Should the action stop the moment I let go?

- Yes → momentary (a doorbell rings only while pressed)
- No, it should stay on until I press again → self-locking (a light switch stays on)

STEPS IN BUILDING YOUR CIRCUIT

A messy breadboard is hard to check and even harder to fix. If you build every circuit the same way, you'll spot mistakes faster, and so will anyone helping you.

The two parts of the breadboard

- Power rail: the long rows along the top and bottom edges, marked red (+) and blue (–). This is where power comes in and gets shared.
- Body: the main area in the middle, where you place your components and make your connections.

Reading positions: letter, then number

The body has letters for rows (A to J) and numbers for columns (1, 2, 3, …). You name a hole by its letter first, then its number, like A30.

Use these labels to place components and to describe your circuit to others. "Put the LED's anode in C12" is much clearer than "put it near the middle."

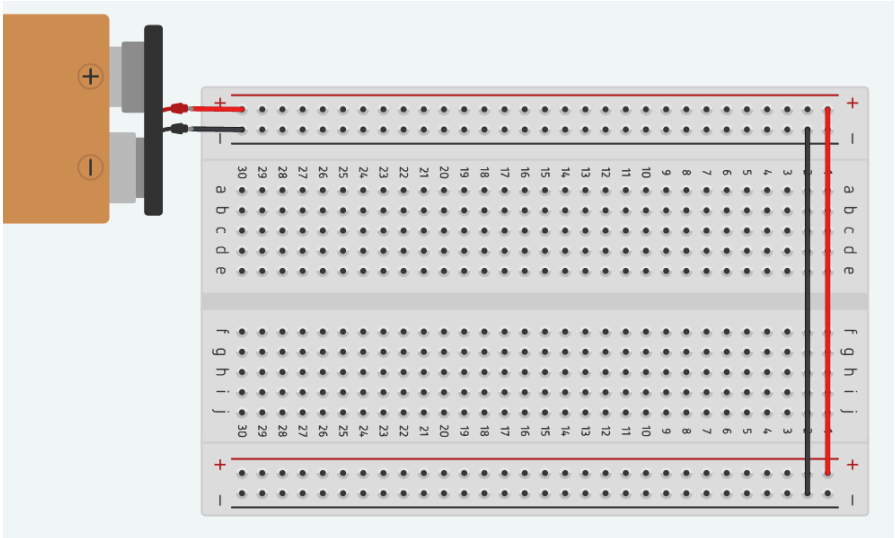
Remember how the body is connected (E03): holes in the same numbered column on the same side of the center gap are connected. A30 to E30 are connected to each other. F30 to J30 are a separate group, across the gap.

Initial Configuration

Most circuits start with this same setup:

1. Connect the battery to the positive (+) and negative (–) rails at the top left of the breadboard.
2. Connect the top right rails to the bottom right rails, matching polarity: top + to bottom +, top – to bottom –.

Now both the top and bottom rails have power, so you can reach positive and negative from anywhere on the board. Start with this configuration most of the time.



Connect your components: Top to Bottom, Left to Right

Place and connect components in order, top to bottom and left to right, the same way you read a page. This keeps your circuit organized. You (and your facilitator) can follow it like reading, and it's easy to see where each part of the circuit begins and ends.

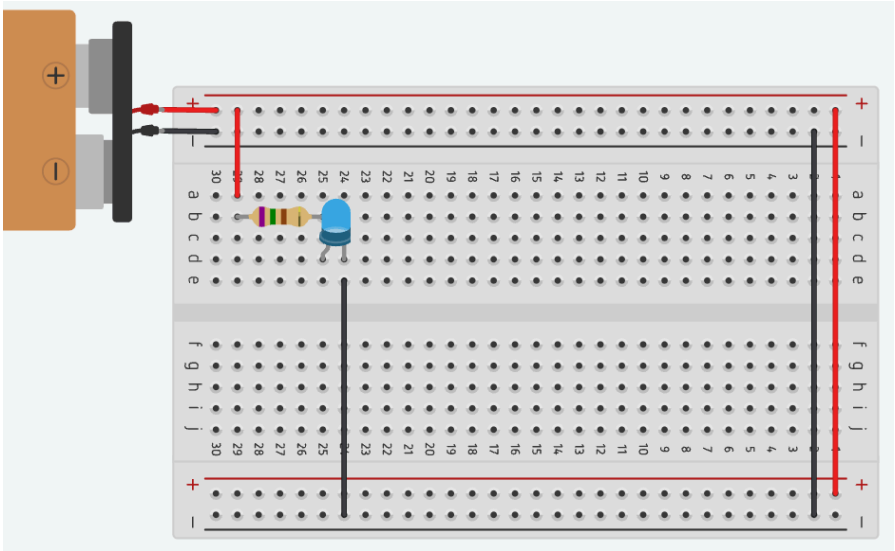
Follow the Resistor First convention

Place the resistor on the positive side of the component:

Power (+) → Resistor → LED anode (long leg) → LED cathode (short leg) → GND (–)

The LED's cathode then connects straight to the negative rail.

Note: A resistor on the negative side also works, because in a series circuit the same current flows through every component. You may see it that way in other guides. At Acamad, we use Resistor First so every circuit is built and checked the same way.



Tool: The Finger Trace (Troubleshooting)

When your circuit doesn't work:

1. Put your index finger on the positive power rail.
2. Follow the connection to the first component. Imagine the current flowing through it.
3. Keep tracing from one component to the next, the way current would flow, until you reach the negative rail.
4. At each component, check:
    - Is it in the right hole (letter and number)?
    - Is it connected to the next component through the same column?
    - Is the polarity correct (anode toward positive, cathode toward negative)?
5. Make sure every component that needs both polarities has a positive and a negative connection. If your finger can't get from + to – through a component, current can't either.

Example: You trace from the + rail to the resistor in B10, then to the LED anode in C10, then the LED cathode in C11... and column 11 never reaches the – rail. Current has nowhere to go, so the LED stays off. Add a jumper wire from column 11 to the negative rail.

04 Introduction to Arduino Programming

ARDUINO HARDWARE

In Section 03, a person controlled the circuit by pressing buttons and turning knobs. With a microcontroller, code controls it. This is the robot's control system: its brain.

An Arduino board is built around a microcontroller, a small computer on a chip that runs your program. Key parts of the board:

- USB port: connects the board to your computer for power and uploading code
- USB-to-serial chip: handles communication between the computer and the microcontroller
- Microcontroller: runs your program
- Digital pins: for digital input and output
- Analog pins (A0, A1, ...): for reading analog input
- Power pins: 5V, 3.3V, GND (ground), and VIN

BASIC OPERATION

Three things to know about how an Arduino runs:

1. Most Arduino boards are designed to run one program at a time on the microcontroller.
2. The program starts running as soon as the board is powered.
3. The program runs incredibly fast, generally within microseconds, unless you tell it to slow down. (The exact speed depends on the size of the program and how long the microcontroller takes to execute it.)

That third point matters. If you blink an LED with no pause, it switches so fast that it looks like it's always on. That's why we use delay().

ARDUINO IDE AND SKETCHES

The Arduino IDE is the software you use to write code and upload it to the board. Download it from www.arduino.cc/en/software.

A sketch is the file you write your program in. It has the .ino extension and is always stored in a folder with the same name.

Program structure
Every Arduino program needs at least two functions:

void setup()

- Runs only once, when the Arduino is powered on or reset.
- This is where you configure things, like setting a pin as input or output, setting the baud rate of serial communication, or starting a library.

void loop()

- Runs over and over for as long as the board has power.
- This is where your main program goes, like turning a lamp on or off based on an input, or taking a sensor reading every few seconds.

BLINK

Tool: Uploading a Sketch

1. Prepare your circuit. Connect an LED and a resistor using the 5V, GND, and D13 (digital pin 13) pins.
2. Have your circuit checked by a facilitator before continuing.
3. Create a new sketch: File → New Sketch (Ctrl+N).
4. Write the code.
5. Verify your sketch (the ✓ button). This checks your code for errors.
6. Choose the correct board and port: Select Other Board and Port, then pick your board and the port it's connected to.
7. Upload your sketch (the → button).

The code

```
void setup() {
  pinMode(13, OUTPUT);
}

void loop() {
  digitalWrite(13, HIGH);
  delay(1000);
  digitalWrite(13, LOW);
  delay(1000);
}
```

What each line does

- pinMode(13, OUTPUT); sets pin 13 to behave as an output. It goes inside setup() because it only needs to happen once.
- digitalWrite(13, HIGH); sends a HIGH signal to pin 13, which turns the LED on.
- delay(1000); pauses the program for 1,000 milliseconds (1 second).
- digitalWrite(13, LOW); sends a LOW signal to pin 13, which turns the LED off.

- Then loop() starts over, so the LED keeps blinking: on for 1 second, off for 1 second.

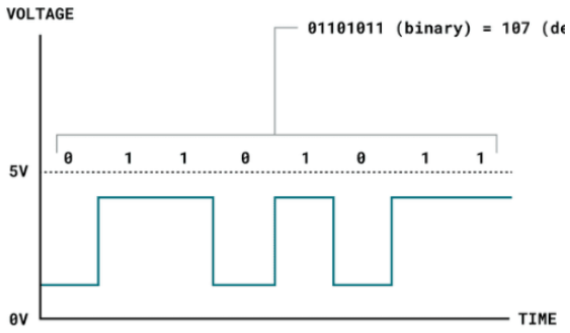
ELECTRONIC SIGNALS: ANALOG & DIGITAL

Sensors and actuators talk to the Arduino in one of two “languages.” You need to know which one each component uses.

Digital signals

A digital signal has only two states: 0 or 1, read as LOW or HIGH. Think of a light switch: it’s either off or on.

- digitalRead(pin) reads the state of a digital pin (HIGH or LOW).
- digitalWrite(pin, value) writes a HIGH or LOW state to a digital pin.
- Examples: a push button (pressed or not), an LED that’s simply on or off.

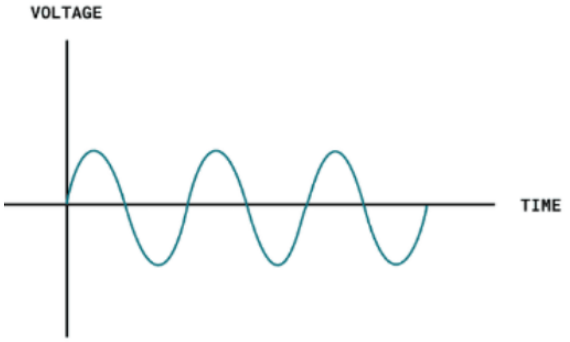


Analog signals

An analog signal can take any value within a range. On an Arduino, that range is usually 0–5 V (or 0–3.3 V on some boards). Think of a dimmer: it can be anywhere between off and full brightness.

- analogRead(pin) reads the value from an analog pin and returns a number from 0 to 1023.
- analogWrite(pin, value) writes a value from 0 to 255 to a pin.
- Examples: a potentiometer (any position between its ends), an LED that fades.

Note: analogWrite() only works on pins marked with a ~ symbol on the board, such as pin 9.



Tool: Digital or Analog?

Ask: Does this only have two states, or can it be anywhere in between?

- Two states → digital
- Anywhere in a range → analog

READING CODE: POTENTIOMETER-CONTROLLED LED

Being able to read code and figure out what it does is just as important as writing it.

```
int potValue = 0;

void setup() {
  pinMode(9, OUTPUT);
}

void loop() {
  potValue = analogRead(A0);
  analogWrite(9, potValue / 4);
  delay(10);
}
```

What it does: It reads the value of a potentiometer and adjusts an LED's brightness based on it.

What each line does

- int potValue = 0; creates a variable called potValue to store a whole number, starting at 0.
- pinMode(9, OUTPUT); sets pin 9 (where the LED is connected) as an output.
- potValue = analogRead(A0); reads the potentiometer on analog pin A0 and stores the result (0 to 1023).
- analogWrite(9, potValue / 4); sets the LED's brightness on pin 9.
- delay(10); pauses for 10 milliseconds before reading again.

Why divide by 4? analogRead() gives a value from 0 to 1023, but analogWrite() only accepts 0 to 255. Dividing by 4 scales the range to fit: 1023 ÷ 4 ≈ 255.